Python -

Data Types & Variables-

Basic Data Types

Type	Example	Description
int	x = 10	Integer numbers
float	y = 3.14	Decimal numbers
str	name = "Alice"	Text or string
bool	flag = True	True or False

String Operations

```
msg = "Python"
print(msg.lower())      # 'python'
print(msg.upper())      # 'PYTHON'
print(msg[0])           # 'P'
print(msg[-1])          # 'n'
print(msg[0:3])         # 'Pyt'
print("Py" in msg)      # True
print(f"Hello {msg}")   # f-string formatting
```

Operators & Expressions

Arithmetic Operators

Operator	Example	Result
+	5 + 3	8
-	05-Mar	2
*	5 * 3	15
/	05-Feb	2.5
//	5 // 2	2 (floor division)
%	5 % 2	1 (remainder)
**	2 ** 3	8 (power)

Comparison Operators

Operator	Example
==	5 == 5
!=	5 != 3
>	5 > 3
<	5 < 3
>=	5 >= 5

Conditionals & Loops

If / Elif / Else-

if Statement

- The if statement is used to check a condition.
- If the condition is True, the block of code under it runs.

elif Statement

- Short for “else if”.
- It lets you check another condition if the previous if was False.

Example -

```
age = 18
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teen")
else:
    print("Child")
```

Nested Conditions

Definition:
A nested condition means putting one if statement inside another if statement.
It allows you to check multiple conditions in stages — the inner condition is only tested if the outer one is true.

Example-

```
x = 10
if x > 0:
    if x % 2 == 0:
        print("Positive even")
```

For Loops :

Definition:

A for loop is used to repeat a block of code a specific number of times.
It’s often used to iterate (go through) items in a sequence like a list, string, tuple, or range of numbers.

```
for i in range(5):
    print(i)  # 0 1 2 3 4

fruits = ["apple", "banana", "cherry"]
for f in fruits:
    print(f)
```

Range variations:

```
range(5)        # 0 to 4
range(2, 6)     # 2 to 5
range(0, 10, 2)  # 0, 2, 4, 6, 8
```

While Loops

Definition -

A while loop is used to repeat a block of code as long as a condition is True.When the condition becomes False, the loop stops.

Example -

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Loop Control

```
for i in range(10):
    if i == 3:
        continue   # skip 3
    if i == 8:
        break      # stop loop
    print(i)
else:
    print("Loop finished!")
```

1. Lists, Tuples, Sets, Dictionaries

- Lists (Ordered, Mutable)

Definition: A list is an ordered, changeable (mutable) collection of items. You can add, remove, or modify elements in a list.

```
fruits = ["apple", "banana", "cherry"]

# Indexing
print(fruits[0])      # apple
print(fruits[-1])     # cherry
```

Tuples (Ordered, Immutable)

A Tuple is a collection of items that is:

- Ordered → items have a fixed position (index)
- Immutable → items cannot be changed after creation

```
t = (10, 20, 30)
print(t[1])         # 20
print(t.count(10))   # 1
print(t.index(30))   # 2
```

Dictionaries (Key-Value Pairs)

Definition :

A Dictionary in Python is a collection of key-value pairs. Each key is unique and is used to access its value.

```
person = {"name": "Alice", "age": 25, "city": "NYC"}

print(person["name"])        # Alice
print(person.get("age"))     # 25
```

Pandas

Pandas is mainly used for data analysis and data manipulation — especially with tables (rows & columns).

- **Importing Pandas**

```
import pandas as pd
```

- **Creating Data Structures**

Create a Series (1D):

```
s = pd.Series([1, 2, 3, 4])
```

Create a DataFrame (2D):

```
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35]}
df = pd.DataFrame(data)
```

- **Selecting Data**

```
df["Name"]          # Select one column
df[["Name", "Age"]]  # Select multiple columns
df.iloc[0]          # Row by index
df.loc[0, "Name"]    # Specific cell (row 0, column 'Name')
```

- **Filtering Data**

```
df[df["Age"] > 25]
df[(df["Age"] > 25) & (df["Name"] == "Bob")]
```